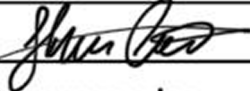


PROYECTO INTEGRADOR DE COMPETENCIAS II

TÍTULO: Desarrollo de un modelo de brazo robótico didáctico de cuatro grados de libertad y efector intercambiable

Autor 1: <i>Pereira Hector</i>	Autor 2: <i>Leuna Mateo</i>	Autor 3: <i>Rossi Priscila</i>
Firma: 	Firma: 	Firma: 
Docentes encargados:		Fecha de Entrega:
Hippa R., Eguia L., Campero C., Díaz M.,		07/09/2025

Índice

1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Diagrama de Ishikawa	3
1.3. Solución propuesta	4
1.4. Objetivo General	4
1.5. Objetivos específicos	4
2. Fundamento técnico-conceptual	4
2.1. Análisis de mercado	4
2.1.1. GrabCAD	5
2.1.2. Robodacta	5
2.2. Diseños considerados	5
2.2.1. Criterios de selección	6
2.3. Sustento técnico de la solución conceptual	7
2.3.1. Actuadores	7
2.3.2. Estructura	9
2.3.3. Fuente de alimentación	10
2.3.4. Microcontrolador	10
2.3.5. Ecuaciones y parámetros relevantes	11
2.3.6. Aspectos electrónicos	14
2.4. Interfaz gráfica de control	14
2.4.1. Interfaz humano-máquina (HMI)	14
2.4.2. Justificación de una interfaz gráfica de control	15
2.4.3. Arquitectura de comunicación entre la aplicación y el microcontrolador	15
2.4.4. Modos de operación del sistema de control	16
2.4.5. Concepto de grabación y reproducción de trayectorias	16
2.4.6. Consideraciones de seguridad en interfaces gráficas de control	16
3. Desarrollo de la solución	16
3.1. Alcance del proyecto	16
3.2. Criterios de aceptación	17
3.3. Segmentación de proyecto	17
3.3.1. Diagrama de Gantt	18
3.4. Materiales	18
3.5. Métodos de fabricación	19
3.5.1. Tolerancias de fabricación	19
3.6. Diseño y fabricación	20
3.6.1. Articulaciones desmontables	20
3.6.2. Electroimán	22
3.7. Fuente de alimentación	23
3.7.1. Controlador espejo	23
3.8. Código microcontrolador	23
3.9. Electrónica de control	23
3.10.	23

A. Apéndice	26
A.1. Recursos del proyecto	26
A.1.1. Google Drive	26
A.1.2. Repositorio en GitHub	26
A.2. Inspiraciones	26
A.3. Código para el microcontrolador	29
A.3.1. Módulo UART: Inicialización y telemetría de servos	29
A.3.2. Parser de comandos UART (MODE, MAG, GON/GOFF, S1..S4)	29
A.3.3. Algoritmo de suavizado incremental (SLEW) para los servos	30
A.3.4. Generación de PWM por software (bit-banging) usando Timer1	31
A.3.5. Control del electroimán (MAG) con feedback mediante buzzer	31
A.3.6. Máquina de estados del buzzer para patrones de beeps	32
A.3.7. Antirrebote por software de los botones físicos	32
A.3.8. Bucle principal del firmware (control, PWM, telemetría)	33
A.4. Código para la interfaz de usuario	33
A.4.1. Módulo de conexión y comunicación serial (UART)	33
A.4.2. Módulo de interpretación de mensajes del firmware (Parser UART)	34
A.4.3. Envío progresivo de comandos desde los sliders (Rate Limiter)	34
A.4.4. Algoritmo de centrado suave de los cuatro servos	35
A.4.5. Grabación periódica de la trayectoria (Logger de poses)	36
A.4.6. Reproducción de trayectorias con prealineado suave	36
A.4.7. Paso discreto de reproducción de la trayectoria	37
A.4.8. Control del electroimán vía UART	38
A.4.9. Visualización 2D del brazo robótico (HUD cinemático)	38
A.4.10. Módulo de interpretación de mensajes del firmware (Parser UART)	39

1. Introducción

1.1. Planteamiento del problema

La robótica está cada vez más involucrada en el mercado laboral, lo que hace que la educación no pueda seguir el ritmo de evolución de tecnologías de producción cada vez más prevalentes. Por ello, se busca fomentar la inclusión de la robótica en diferentes niveles educativos: inspirando a los más jóvenes, introduciendo a los estudiantes en educación secundaria, y permitiendo que los más avanzados puedan profundizar en el área.

En este contexto, surge la necesidad de un modelo de brazo robótico accesible, fácil de ensamblar, de bajo costo y con un diseño sencillo que sea adecuado para usuarios inexpertos o jóvenes. Este modelo debe ser, además, duradero, mantenible y fácil de reparar, para garantizar su uso a largo plazo. Adicionalmente, se plantea que el manipulador sea modular, con la posibilidad de modificación y expansión, permitiendo a los estudiantes involucrarse en proyectos más complejos y adaptarlos a otros sistemas o tecnologías.

El objetivo final es ofrecer un manipulador robótico que pueda adaptarse a diversas aplicaciones y demostraciones, fomentando el aprendizaje en robótica y automatización desde etapas tempranas de formación, inspirando a los estudiantes y acercándolos a los procesos industriales actuales.

1.2. Diagrama de Ishikawa

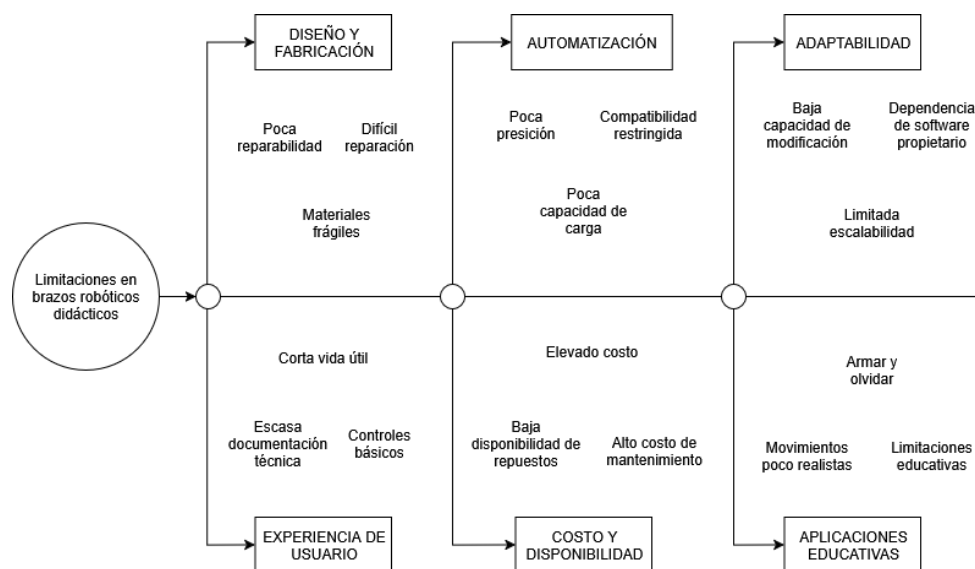


Figura 1: Diagrama de Ishikawa (Fuente: elaboración propia)

En el diagrama se identifican seis ramas principales que agrupan las limitaciones más comunes de los brazos robóticos didácticos disponibles en el mercado. Cada una de ellas aborda un conjunto de problemáticas específicas:

- Diseño y fabricación: engloba aspectos relacionados con la construcción del brazo, los materiales empleados y la facilidad de reparación o mantenimiento.
- Automatización: considera las limitaciones en la precisión, capacidad de carga, compatibilidad y posibilidades de integración con otros sistemas de control.

- Adaptabilidad: se refiere a la capacidad de modificar, escalar o personalizar el brazo, así como a la dependencia de software propietario que restringe su flexibilidad.
- Experiencia de usuario: abarca las dificultades que enfrentan los usuarios en el manejo del brazo, tales como controles limitados, documentación insuficiente y vida útil reducida.
- Costo y disponibilidad: contempla el precio de adquisición, el costo de mantenimiento y la dificultad para conseguir repuestos o componentes compatibles.
- Aplicaciones educativas: analiza las limitaciones de los kits en contextos formativos, como la escasa realismo de los movimientos y el alcance reducido de sus usos pedagógicos.

1.3. Solución propuesta

Un brazo robótico didáctico a escala de 4 grados de libertad, con efector intercambiable: electroimán (para piezas ferromagnéticas) y garra (para piezas no magnéticas). Estructura paramétrica cortada en MDF (3.2–10 mm), tornillería estándar y electrónica de fácil reposición. Control con ESP32/ATmega328P, modos manual/automático, GUI en Python (Tkinter/PyQt), y documentación abierta para replicación.

1.4. Objetivo General

Desarrollar un modelo de brazo robótico didáctico de 4 grados de libertad, seguro, portable y accesible, con efectores electroimán/garra intercambiables.

1.5. Objetivos específicos

1. Investigar materiales y componentes para la estructura y mecanismos del dispositivo
2. Diseñar la estructura mecánica del brazo robótico de 4 grados de libertad.
3. Desarrollar un sistema de actuación con servomotores que permita manipular cargas útiles dentro del contexto educativo/demostrativo.
4. Implementar un sistema que permita un cambio rápido del efector del robot
5. Diseñar planos eléctricos y esquemas programáticos del funcionamiento interno del dispositivo
6. Desarrollar una interfaz gráfica de usuario (GUI) que permita el control manual y automático del brazo.

2. Fundamento técnico-conceptual

2.1. Análisis de mercado

Se realizó una investigación de soluciones existentes con el fin de identificar referentes, buenas prácticas de diseño y alcances realistas para un kit didáctico. Los modelos y enlaces tomados como referencia se listan en el apéndice A.2.

2.1.1. GrabCAD

GrabCAD es una plataforma comunitaria donde profesionales y estudiantes comparten modelos 3D y ensamblajes CAD. Permite publicar, buscar y descargar archivos en formatos habituales (STEP/IGES, STL, SLDPRPT), así como previsualizar, documentar y comentar mejoras. En esta revisión se localizaron varios brazos robóticos con distintos esquemas de actuación, útiles para comparar arquitecturas mecánicas, distribución de masas, soluciones de juntas y estrategias de fabricación.

2.1.2. Robodacta

Robodacta es una empresa mexicana dedicada a kits de robótica educativa para distintos niveles. Sus propuestas muestran un equilibrio práctico entre costo, montaje y funcionalidad, valioso como punto de referencia. Para los objetivos de este proyecto, sin embargo, su adopción directa presenta límites: la documentación y el software son principalmente propietarios, lo que reduce la posibilidad de adaptación y reutilización abierta, y la importación añade costos y tiempos que afectan la accesibilidad en el contexto local.

2.2. Diseños considerados

Todas las soluciones involucran un brazo robótico de 4 articulaciones controladas por un Arduino. Las variaciones en los modelos se pueden clasificar en como se manejan las articulaciones y los actuadores utilizados. Las clasificaciones son:

- **Servomotores MG996R** Un modelado directo y más convencional del brazo robótico, utilizando un servomotor por articulación y colocado directamente en la misma. Apuntando el diseño a la simplicidad, en fácil ensamblado y la reparabilidad, utilizando una combinación entre corte láser en de mdf (en mayor medida) e impresión 3d.
- **Servomotores MG996R con eslabones.** Modelo implementado con servomotores pero optimizando la distribución de peso alojando a los mismos lo más cerca del eje de rotación de cada articulación y transmitiendo el movimiento mediante eslabones y palancas, buscando mejorar la eficiencia mecánica. El diseño se vuelve más vistoso y potencialmente útil para instruir en el uso de eslabones y mecanismos simples, sin embargo aumenta la complejidad del diseño llevando mayor cantidad de partes móviles, y disminuyendo la facilidad de ensamble y reparación.
- **Actuadores lineales tornillo—tuerca (motor DC con reductora).** Ofrecen alta fuerza y buen costo por actuador, y pueden estar enfocados a niveles educativos superiores por la implementación de controles PID con sensado de ángulo y calibración manual, entre otras cosas. Sin embargo, el mecanismo es más lento y menos preciso; requiere encoders o potenciómetros y finales de carrera para conocer el ángulo articular. Además, el diseño de eslabones y articulaciones se torna más complejo, y los mecanismos de codificación de posición son más propensos a *interferencias/ruido* y se convierten en otro posible punto de fallo. El juego mecánico (backlash) y la fricción, además del desgaste de los potenciómetros pueden degradar la repetibilidad.
- **Actuadores lineales con asistencia hidráulica (jeringas).** Siendo la alternativa más atractiva, pero también la más compleja. Añade una capa de transmisión hidráulica sobre el actuador lineal. A cambio, ofrece una *relación fuerza—peso* muy favorable (bajo

peso en el extremo móvil frente a la fuerza aplicable). No obstante, introduce menor velocidad, necesidad de purgado/cebado, riesgo de fugas y mayor mantenimiento; el control (válvulas, amortiguación) eleva la complejidad significativamente. Al igual que en la opción de tornillo—tuerca, la posición debe codificarse (p. ej., potenciómetros/encoders lineales), añadiendo potenciales puntos de fallo.

2.2.1. Criterios de selección

- Distribución de peso: mide la capacidad del sistema de actuación para ubicar la masa de los actuadores cerca de la base o de los ejes principales, reduciendo así el par requerido en las articulaciones. Una buena distribución de peso incrementa la estabilidad y mejora la eficiencia energética del manipulador.
- Simplicidad: evalúa el grado de complejidad en el diseño, ensamblaje y operación del mecanismo. Un sistema más simple resulta más accesible para estudiantes y usuarios inexpertos, reduciendo la curva de aprendizaje y los posibles errores durante el montaje o uso.
- Costo: considera tanto el precio inicial de adquisición de los actuadores como el de los componentes asociados (accesorios, controladores, sensores, etc.). Este criterio es clave para mantener la accesibilidad del brazo robótico en entornos educativos.
- Reparabilidad: analiza la facilidad para reemplazar piezas dañadas o desgastadas, así como la disponibilidad de repuestos en el mercado. Una buena reparabilidad asegura la continuidad en el uso y la sostenibilidad a largo plazo del prototipo.
- Durabilidad: se refiere a la resistencia del sistema de actuación frente al desgaste mecánico, la fatiga de materiales y el uso prolongado. Los mecanismos más duraderos requieren menos mantenimiento y ofrecen mayor confiabilidad.
- Capacidad de carga: mide la fuerza máxima que el sistema puede ejercer en el efector final, lo cual determina el rango de piezas u objetos que el brazo robótico puede manipular. En el contexto didáctico, se prioriza un equilibrio entre capacidad de carga y ligereza del diseño.
- Velocidad: hace referencia al tiempo de respuesta y a la rapidez con la que el actuador puede ejecutar un movimiento. Una mayor velocidad contribuye a realizar demostraciones más dinámicas y a simular con mayor realismo procesos industriales.
- Precisión: evalúa la exactitud con la que el sistema puede alcanzar y mantener una posición deseada. Este atributo es fundamental para la enseñanza de conceptos de control, automatización y cinemática, ya que un mayor nivel de precisión permite ejecutar trayectorias repetibles y confiables.

Cuadro 1: Evaluación comparativa de mecanismos de actuación

Atributo	Peso	Servom.	Eslabones	Actuadores lin.	Pistones hidr.
Distribución de peso	1	6	8	9	10
Simplicidad	3	10	6	5	4
Costo	3	9	8	7	7
Reparabilidad	2	9	7	6	5
Durabilidad	2	8	5	4	3
Cap. de carga	2	7	7	8	9
Velocidad	1	8	8	5	4
Precisión	1	8	7	6	5
Total ponderado		127	103	92	86

Tomando en cuenta los criterios especificados con sus respectivos pesos asignados, se llega a la conclusión que un diseño con servomotores articulado es el más adecuado para llevar a cabo los objetivos planteados.

Lineamientos de diseño adoptados

- Enfocar el diseño en la simplicidad de montaje, reduciendo la cantidad de componentes.
- Reducir cantidad de elementos propensos a desgaste/fallas, planeando adecuadamente el movimiento y las interacciones físicas entre los materiales del dispositivo.
- Tomar consideración de las tolerancias de fabricación en elementos deslizantes.
- Mantener la modularidad e intercambiabilidad del efector (electroimán/garra) y el diseño paramétrico para distintos espesores de MDF.

Los lineamientos de diseño se centran en garantizar un montaje sencillo y confiable, minimizando la cantidad de componentes y puntos de desgaste. Se consideraron las tolerancias de fabricación en elementos móviles, y se mantuvo un enfoque modular y paramétrico que permite la intercambiabilidad del efector y la adaptación a distintos espesores de MDF.

2.3. Sustento técnico de la solución conceptual

2.3.1. Actuadores

Al comparar el MG996R con el SG90 (Oracid (2013) y TSE (2021)) se nota enseguida que están pensados para usos muy diferentes. El MG996R es mucho más fuerte: puede llegar a un par de hasta 10 kgf·cm, mientras que el SG90 apenas alcanza 0.7 kgf·cm. Esa diferencia hace que el primero sea ideal para mover piezas más pesadas o para darle solidez a un brazo robótico, mientras que el segundo queda limitado a proyectos pequeños y livianos.



Figura 2: Comparación de tamaños entre servomotores SG90, MG90S y MG996R. Fuente: elaboración propia

En tamaño y peso también se distinguen bastante. El SG90 es muy compacto, con solo 9 gramos, frente a los 55 gramos del MG996R. Esa ligereza lo vuelve práctico cuando no hay mucho espacio o cuando se busca mantener la estructura lo más liviana posible, aunque lógicamente no tiene la misma capacidad de carga. En cuanto a velocidad, el SG90 responde un poco más rápido ($0.1 \text{ s}/60^\circ$ frente a $0.17 \text{ s}/60^\circ$ a 4.8 V en el MG996R), pero esta diferencia no suele ser decisiva si lo que se necesita es fuerza y confiabilidad más que rapidez.

Otra diferencia clave está en los materiales. El MG996R usa engranajes metálicos y un sistema de doble rodamiento de bolas, lo que le da más precisión y resistencia al desgaste. En cambio, el SG90 usa engranajes plásticos, que son más frágiles y tienden a deteriorarse con el uso. Eso sí, esta robustez del MG996R viene con un costo: su consumo de corriente es bastante más alto (puede llegar a 2.5 A en stall), mientras que el SG90 consume mucho menos y puede funcionar sin problemas con fuentes pequeñas.

Cuadro 2: Matriz de selección entre servos MG996R y SG90

Atributo	Peso	Servo MG996R	Servo SG90
Fuerza (torque)	4	10	3
Tamaño (dimensiones)	2	6	9
Peso (masa)	2	5	9
Precio	2	7	9
Durabilidad / materiales	3	9	5
Precisión	2	7	6
Velocidad ($\text{s}/60^\circ$)	1	6	9
Corriente (menor=mejor)	2	4	8
Total ponderado		131	118

Basado en el resultado de la matriz de selección, el MG996R es la mejor opción cuando se busca un servo confiable, fuerte y duradero para un brazo robótico educativo que tenga que manipular piezas con cierta carga. El SG90, en cambio, es más adecuado para pruebas preliminares, proyectos sencillos o demostrativos, donde lo que importa es que sea barato, liviano y fácil de

integrar. (Tower Pro (2015))

2.3.2. Estructura

Para la fabricación del brazo se evaluaron tres alternativas: (i) estructura íntegramente en MDF mediante corte láser, (ii) piezas 100 % impresas en 3D (PLA) y (iii) enfoque híbrido, utilizando MDF como material estructural principal e interfaces de PLA en zonas de fricción y articulaciones. En el contexto del taller, el MDF ofrece una ventaja clara en *tiempo de iteración*: permite realizar ajustes dimensionales rápidos (re-cortes sucesivos) y validar encastres sin esperar horas de impresión Xometry (2022). Además, su costo por unidad de superficie es bajo y la documentación técnica del MDF como material de ingeniería está ampliamente consolidada Ross y Laboratory (2021).

Ahora bien, el MDF presenta limitaciones en contactos móviles: con fricción sostenida tiende a deshilacharse y perder tolerancia, lo que afecta la repetibilidad de las uniones; de hecho, la fricción en MDF no laminado se reporta relativamente alta frente a otras chapas y tableros Kukla, Wargula, y Byszczanik (2021). Allí el PLA aporta valor: permite generar *interfaces* con mejor control de tolerancias, superficies de contacto más estables y geometrías que no son viables con corte láser (chafanes, alojamientos, casquillos, topes, etc.), con evidencia tribológica específica sobre el comportamiento de PLA impreso (influencia de textura, parámetros de impresión y lubricación) Thirunavukkarasu y cols. (2022).¹

La comparación cuantitativa se resume en la matriz de selección (véase Tabla 3), ponderando criterios como tiempo de fabricación, costo, facilidad de retrabajo, precisión/tolerancias, comportamiento a la fricción, rigidez estructural, peso y complejidad de ensamblaje.

Cuadro 3: Matriz de selección: MDF vs Impresión 3D vs Híbrido (MDF+PLA)				
Atributo	Peso	MDF	PLA	MDF+PLA
Tiempo de fabricación (menor=mejor)	3	9	3	8
Costo de materiales (menor=mejor)	2	9	5	8
Facilidad de iteración/retrabajo	3	10	4	9
Precisión / tolerancias	2	6	8	8
Comportamiento a la fricción	3	3	8	8
Rigidez / resistencia estructural	3	7	6	8
Peso total (menor=mejor)	1	6	7	7
Complejidad de ensamblaje (menor=mejor)	1	8	6	6
Disponibilidad de recursos (UTEC)	1	10	6	9
Acabado / ajuste final	1	6	7	7
Total ponderado		147	115	160

Con base en esa matriz, se adopta un enfoque **híbrido (MDF + interfaces PLA)**. La mayor parte de la carga estructural se resuelve con MDF, aprovechando su rapidez y economía en iteraciones; las *interfaces de PLA* se emplean en articulaciones y superficies en contacto para asegurar durabilidad y estabilidad dimensional. Este esquema balancea costo y tiempos de fabricación con el desempeño mecánico requerido, y reduce riesgos de reprocesos largos propios

¹En zonas PLA-PLA se recomienda lubricante seco y orientar las capas para minimizar desgaste.

de una solución 100 % impresa. Como consideraciones prácticas, se recomienda sellar el MDF para humedad, utilizar lubricante seco en contactos PLA-PLA, controlar la orientación de capas en piezas impresas y diseñar encastres reemplazables en las zonas de mayor desgaste.

2.3.3. Fuente de alimentación

Para la alimentación del brazo se evaluaron cuatro alternativas: PC (fuente ATX reciclada), una fuente conmutada de 5 V y 10 A tipo “LED”, varias fuentes genéricas de 5 V y 2 A distribuidas por carga, y una fuente de laboratorio regulable. La fuente de PC ofrece alto margen de corriente en 5 V, protecciones integradas habituales y muy buena disponibilidad a bajo costo, a cambio de mayor volumen y la necesidad de habilitar PS_ON, además de considerar el límite combinado 3.3 V + 5 V. La fuente LED es compacta y sencilla de cablear, pero suele tener menos margen ante picos simultáneos de servos y protecciones más básicas. La opción de múltiples fuentes genéricas permite aislar cargas, aunque complica el cableado, incrementa los puntos de falla y no comparte bien los picos intermitentes entre líneas separadas. La fuente de laboratorio ofrece excelente regulación y medición, pero no resulta práctica para un kit didáctico autónomo por costo, tamaño y dependencia de un equipo externo.

La comparación cuantitativa se resume en la matriz de selección (véase Tabla 4), donde se ponderan corriente y picos, disponibilidad y costo, integración, regulación y rizado, protecciones, portabilidad, EMI, escalabilidad y confiabilidad en una escala de 1 a 10.

Cuadro 4: Matriz de selección de fuente de poder

Atributo	Peso	PC	LED	Genéricas	Fuente de lab.
Corriente continua y picos	3	9	7	6	8
Disponibilidad + costo	2	10	7	8	3
Facilidad de integración/ensamble	2	6	9	4	5
Regulación y rizado	2	8	6	7	9
Protecciones (OCP/OVP/OTP, etc.)	2	9	6	7	9
Portabilidad / formato	1	5	7	6	3
Ruido eléctrico/EMI	1	7	6	6	8
Escalabilidad (margen futuro)	2	9	6	5	7
Confiabilidad global	2	8	7	5	8
Total ponderado		139	116	102	117

Con base en esa matriz (PC = 139, LED = 116, Genéricas = 102, Fuente de lab. = 117), se elige la fuente ATX de PC porque ofrece el mejor equilibrio entre margen de corriente para picos de servos, protecciones, escalabilidad y disponibilidad/costo para un kit educativo. Como guía de implementación: no usar la línea +5VSB, puentear PS_ON a masa para encendido, distribuir 5 V con fusibles por rama, colocar capacitores de reserva cerca de los servos y realizar conexión de masa en estrella para reducir caídas y ruido.

2.3.4. Microcontrolador

Se consideraron tres controladores para el prototipo: Arduino UNO, Arduino NANO y ESP32. El Arduino UNO fue el más disponible en el taller y es el entorno con el que el equipo ya ha trabajado en otras asignaturas, lo que reduce tiempos de integración y riesgo de bloqueos. Su

ecosistema es estable, la documentación y ejemplos son abundantes y la interfaz a 5 V simplifica el manejo de servomotores y periféricos típicos del proyecto (Arduino (2024b)).

El Arduino NANO ofrece la misma plataforma ATmega328P en un formato más compacto. Mantiene la compatibilidad de librerías, el mapa de pines esencial y el mismo flujo de programación, con la ventaja de ocupar menos espacio en la base del brazo. A cambio, el conector y la disposición de pines pueden resultar menos cómodos en banco y, según versión, la disponibilidad en stock puede ser algo menor que la del UNO (Arduino (2024a)).

La ESP32 agrega mayor capacidad de cómputo y conectividad inalámbrica (Wi-Fi/BLE), pero requiere trabajar a 3.3 V lógicos, puede demandar adaptación de niveles para ciertos periféricos, y su curva de aprendizaje es más pronunciada para el equipo. Además, su disponibilidad local y costo por unidad resultan menos favorables en el contexto educativo planteado (Espressif Systems (2024a)).

La comparación cuantitativa de estos criterios se presenta en la matriz de selección (véase Tabla 5).

Cuadro 5: Matriz de selección de microcontrolador				
Atributo	Peso	UNO	NANO	ESP32
Disponibilidad en UTEC	3	10	9	5
Integración con servos/5 V	3	9	9	6
Ecosistema y soporte en curso	2	10	9	7
Costo	2	8	9	8
Tamaño	1	7	10	8
Rendimiento (CPU/RAM)	1	6	6	10
Conectividad (UART extra/WiFi/BLE)	1	3	3	10
Consumo (menor = mejor)	1	8	9	5
Robustez en banco (taller/educativo)	1	8	7	7
Expansión de E/S y temporizadores	1	8	7	9
Total ponderado		133	132	112

Con base en esa matriz y en la experiencia del equipo, se selecciona Arduino UNO. Es la opción más apropiada por disponibilidad, familiaridad, integración directa a 5 V, ecosistema y costo. El NANO queda como alternativa equivalente cuando el espacio sea crítico sin cambiar el entorno de trabajo. La ESP32 se considera para una revisión futura en la que la conectividad inalámbrica y el mayor rendimiento sean requisitos explícitos.

2.3.5. Ecuaciones y parámetros relevantes

Par y torque

$$\boldsymbol{\tau} = \boldsymbol{r} \times \boldsymbol{F}. \quad (1)$$

- $\boldsymbol{\tau} \rightarrow$ torque (kgf·cm o N·m)
- $\boldsymbol{F} \rightarrow$ fuerza en (kgf o N)
- $\boldsymbol{r} \rightarrow$ distancia del eje al punto de aplicación de la fuerza (cm o m)

Physics LibreTexts (2023)

Potencia eléctrica del servo

$$P_{\text{servo}} = V I . \quad (2)$$

- $P_{\text{servo}} \rightarrow$ potencia consumida por el servo (W)
- $V \rightarrow$ voltaje de alimentación (V)
- $I \rightarrow$ corriente consumida por el servo (A)

OpenStax (2020)

Consumo total del sistema

$$I_{\text{total}} = \sum I_{\text{servos}} + I_{\text{electroiman}} + I_{\text{microcontrolador}} + I_{\text{indicadores}} . \quad (3)$$

- $I_{\text{total}} \rightarrow$ corriente total necesaria (A)
- $I_{\text{servos}} \rightarrow$ corriente de cada servo (A)
- $I_{\text{electroiman}} \rightarrow$ corriente del electroimán (A)
- $I_{\text{microcontrolador}} \rightarrow$ corriente del microcontrolador (A)
- $I_{\text{indicadores}} \rightarrow$ corriente consumida por com componentes indicadores(A)

OpenStax (2022)

Control por PWM

$$\theta = \frac{t_{\text{high}} - t_{\text{mín}}}{t_{\text{máx}} - t_{\text{mín}}} (\theta_{\text{máx}} - \theta_{\text{mín}}) + \theta_{\text{mín}} . \quad (4)$$

- $\theta \rightarrow$ ángulo del servo ($^{\circ}$)
- $t_{\text{high}} \rightarrow$ duración del pulso de control (s)
- $t_{\text{mín}}, t_{\text{máx}} \rightarrow$ tiempos de pulso correspondientes a los extremos del ángulo (s)
- $\theta_{\text{mín}}, \theta_{\text{máx}} \rightarrow$ ángulos extremos del servo ($^{\circ}$)

ServoCity (s.f.); Wikipedia contributors (2025a)

Momento de inercia de las piezas rotantes

$$I = \sum_i m_i r_i^2 . \quad (5)$$

- $I \rightarrow$ momento de inercia ($\text{kg}\cdot\text{cm}^2$ o $\text{kg}\cdot\text{m}^2$)
- $m_i \rightarrow$ masa de cada componente (kg)
- $r_i \rightarrow$ distancia al eje de rotación (cm o m)

OpenStax (2016)

Fuerza del electroimán

$$F_{\text{em}} \approx \frac{B^2 A}{2\mu_0} \text{ con } B \simeq \mu_0 \frac{NI}{g} \Rightarrow F_{\text{em}} \approx \frac{(\mu_0 NI)^2 A}{2g^2}. \quad (6)$$

- F_{em} → fuerza de atracción del electroimán (N)
- μ → permeabilidad del núcleo (H/m)
- N → número de vueltas de la bobina
- I → corriente aplicada (A)
- A → área del núcleo (m²)
- g → distancia entre el núcleo y la pieza ferromagnética (m)

Wikipedia contributors (2025b)

Resistencia de un conductor de cobre

$$R = \rho \frac{\ell}{A}. \quad (7)$$

- R → resistencia eléctrica del segmento (Ω)
- ρ → resistividad del material ($\Omega \cdot \text{m}$); para cobre a 20 °C, $\rho \approx 1,68 \times 10^{-8} \Omega \cdot \text{m}$
- ℓ → longitud del conductor (m)
- A → área de la sección transversal (m²); para alambre circular, $A = \pi \left(\frac{d}{2}\right)^2$
- d → diámetro del alambre (m) (*si se usa la expresión de A*)

Physics LibreTexts (2025)

Corrección por temperatura

$$R(T) = R_0 [1 + \alpha (T - T_0)]. \quad (8)$$

- $R(T)$ → resistencia a la temperatura T (Ω)
- R_0 → resistencia a la temperatura de referencia T_0 (Ω), típicamente 20 °C
- α → coeficiente de temperatura del cobre ($^{\circ}\text{C}^{-1}$), $\alpha \approx 3,9 \times 10^{-3}$
- T → temperatura de operación ($^{\circ}\text{C}$)
- T_0 → temperatura de referencia ($^{\circ}\text{C}$)

HyperPhysics (s.f.)

2.3.6. Aspectos electrónicos

El sistema electrónico se concibe para ofrecer control preciso, seguridad y facilidad de mantenimiento. El *Arduino Uno* (ATmega328P) actúa como controlador principal: genera las señales PWM para los servomotores, adquiere las entradas de potenciómetros y pulsadores, y coordina los estados de operación y seguridad. Cada servo MG996R incorpora un fusible en su línea de alimentación para limitar daños ante sobrecorrientes. La distribución de masas separa los retornos de potencia (servos y electroimán) del retorno de lógica, reuniéndose en un punto estrella cercano a la fuente con el objetivo de minimizar caídas y ruido eléctrico.

El electroimán se conmuta con un MOSFET N-canal en configuración de low-side switch (fuente común). Se emplea un IRFZ44N en encapsulado TO-220, cuyas magnitudes relevantes (tensión

$V_{DS,max} = 55$ V, corriente continua admisible del orden de decenas de amperios a 25°C y resistencia de conducción especificada como $R_{DS(on)} \leq 22$ m Ω a $V_{GS} = 10$ V) resultan holgadas para el uso previsto. Dado que la hoja de datos garantiza la baja $R_{DS(on)}$ a $V_{GS} = 10$ V y no a 5 V, el dispositivo no es estrictamente de compuerta lógica; sin embargo, a las corrientes que demanda nuestro electroimán su desempeño es adecuado, y se deja constancia de que una evolución razonable es sustituirlo por un modelo *logic-level* (p. ej., IRLZ44N) para reducir pérdidas y elevar el margen térmico.

La bobina del electroimán está realizada con 20 m de alambre de cobre esmaltado de diámetro $d = 0,5$ mm. Su resistencia en frío se estima con

$$R = \rho \frac{\ell}{A}, \quad \rho_{\text{Cu}} \approx 1,68 \times 10^{-8} \text{ } \Omega \cdot \text{m}, \quad A = \pi \left(\frac{d}{2} \right)^2,$$

lo que arroja aproximadamente $R \approx 1,68$ Ω . Alimentada a 5 V, la corriente esperada es $I_{\text{bob}} \approx 5/1,68 \simeq 3,0$ A y la disipación en la bobina $P_{\text{bob}} \approx I_{\text{bob}}^2 R \simeq 15$ W. Estas magnitudes aconsejan operación intermitente (control de *duty-cycle*) o gestión térmica adicional para mantener temperaturas seguras en uso continuado.

La interfaz de control se completa con una resistencia en serie en la compuerta del MOSFET ($R_g \approx 100$ Ω) para limitar el régimen de carga de la compuerta y un *pull-down* ($R_{pd} \approx 100$ k Ω) que asegura el apagado en reposo. En paralelo con la bobina se dispone un diodo de rueda libre (*flyback*) rápido o Schottky, dimensionado para la corriente de la bobina y con tensión inversa acorde, a fin de suprimir sobretensiones al corte. En la alimentación de 5 V se ubican condensadores de *bulk* cerca de las cargas (típicamente del orden de 470–1000 μF) y desacoples locales de 100 nF, con la finalidad de amortiguar picos debidos a los servomotores y al propio electroimán. Finalmente, el firmware incorpora medidas de protección como límites por *software* en el tiempo de activación del electroimán, vigilancia por *watchdog* y paro de emergencia, coherentes con la filosofía de diseño seguro del conjunto.

2.4. Interfaz gráfica de control

2.4.1. Interfaz humano-máquina (HMI)

En los sistemas robóticos con múltiples grados de libertad, la interacción entre el operador y el sistema representa un elemento crítico para garantizar precisión, seguridad y repetitividad. Las interfaces humano-máquina (HMI, por sus siglas en inglés) permiten que el usuario pueda supervisar y controlar el estado del sistema sin necesidad de interactuar directamente con los niveles más bajos del hardware.

En el contexto de los brazos robóticos, las interfaces gráficas de usuario (GUI) se han consolidado como una solución estándar tanto en entornos industriales como educativos. Estas interfaces no solo facilitan el control manual, sino que también permiten visualizar información en tiempo real, como posiciones angulares, trayectorias, estados de los actuadores y condiciones de funcionamiento.

La utilización de una HMI basada en software presenta ventajas significativas frente a interfaces exclusivamente físicas, tales como:

- Mayor precisión en el ajuste de parámetros de movimiento.
- Visualización clara del estado del sistema en tiempo real.
- Posibilidad de implementar funciones avanzadas como grabación y reproducción de trayectorias.
- Mayor seguridad operativa al reducir la intervención directa del usuario sobre el hardware.

A partir de estos principios, se decidió incorporar una aplicación de escritorio como parte integral del sistema de control del prototipo.

2.4.2. Justificación de una interfaz gráfica de control

La elección de una interfaz gráfica de escritorio frente a otras alternativas, como controles físicos dedicados o paneles externos, se basa en criterios de flexibilidad, escalabilidad y facilidad de desarrollo.

Una aplicación de escritorio permite modificar y ampliar las funcionalidades del sistema sin alterar el hardware, lo que resulta especialmente útil en entornos de prototipado y desarrollo. Asimismo, facilita la implementación de funciones complejas como la automatización de movimientos, la visualización dinámica del estado del sistema y la integración de funciones de seguridad mediante software.

Este enfoque resulta adecuado para proyectos académicos y de investigación, donde se requiere iterar rápidamente sobre el diseño y evaluar distintos comportamientos del sistema sin necesidad de reconstruir la electrónica de control.

2.4.3. Arquitectura de comunicación entre la aplicación y el microcontrolador

El sistema se diseñó bajo una arquitectura distribuida, en la cual las tareas de alto nivel se delegan a una aplicación externa, mientras que el control de bajo nivel permanece en el microcontrolador.

La aplicación es responsable de la generación de consignas de movimiento, la gestión de trayectorias y la interacción con el usuario, mientras que el microcontrolador ejecuta el control de tiempo real de los actuadores. La comunicación entre ambos módulos se realiza mediante un enlace serie asíncrono (UART), utilizando un protocolo textual simplificado.

Esta separación de responsabilidades permite que el sistema mantenga un comportamiento determinístico en el control de los servomotores, al mismo tiempo que ofrece una interfaz flexible y de fácil modificación.

2.4.4. Modos de operación del sistema de control

Desde un punto de vista conceptual, el sistema fue diseñado para operar en dos modos diferenciados:

- **Modo de control manual (control espejo):** en este modo, la prioridad la tiene el controlador físico, y el sistema replica los movimientos realizados por el operador de forma directa.
- **Modo de control por aplicación:** en este modo, las consignas de movimiento se generan desde la interfaz gráfica, permitiendo un control más preciso y la posibilidad de automatizar secuencias.

La existencia de estos modos responde al principio de diseño de redundancia funcional, permitiendo que el sistema continúe siendo operable aun cuando uno de los métodos de control no se encuentre disponible.

2.4.5. Concepto de grabación y reproducción de trayectorias

La posibilidad de registrar y repetir trayectorias es una característica presente en sistemas robóticos industriales, especialmente en aplicaciones de pick-and-place, manipulación y ensamblaje.

Desde el punto de vista teórico, la grabación de trayectorias consiste en muestrear periódicamente el estado del sistema (en este caso, los ángulos articulares) y almacenarlos como una secuencia temporal de estados discretos. Posteriormente, estos estados pueden ser reproducidos respetando el mismo intervalo temporal, logrando la repetición aproximada del movimiento original.

Este enfoque permite analizar el comportamiento dinámico del sistema, optimizar movimientos y facilitar tareas repetitivas sin depender exclusivamente del control manual.

2.4.6. Consideraciones de seguridad en interfaces gráficas de control

En sistemas mecatrónicos, el diseño de la interfaz de control no solo debe enfocarse en la funcionalidad, sino también en la seguridad del equipo y del operador.

Una interfaz bien diseñada debe evitar cambios bruscos de estado, incorporar verificaciones de coherencia en los comandos enviados y garantizar transiciones suaves entre modos de operación. Estos principios se aplicaron en el diseño conceptual de la aplicación de control, priorizando movimientos progresivos al iniciar trayectorias automáticas y limitando los rangos de operación de los actuadores por software.

3. Desarrollo de la solución

3.1. Alcance del proyecto

El proyecto comprende el diseño, construcción y validación de un brazo robótico didáctico a escala. La arquitectura mecánica contempla cuatro grados de libertad (sin contar el efector), con una estructura paramétrica diseñada para corte láser en MDF de espesores entre 3.2 y 10 mm, pero adaptable también a materiales como acrílico o metal. La base será fija y deberá estar contenida dentro de un volumen de $50 \times 50 \times 50$ cm.

El sistema contará con efectores intercambiables, entre ellos un electroimán para la manipulación de piezas ferromagnéticas y una garra mecánica para piezas no magnéticas. Ambos estarán diseñados con un mecanismo de cambio rápido que permita alternar entre uno y otro sin dificultad.

La actuación del brazo se logrará mediante servomotores MG996R, con uno por cada articulación principal. Para optimizar el rendimiento, se implementará una transmisión por eslabones que mantenga la mayor parte de la masa cercana a los ejes de giro.

En cuanto al control y la electrónica, se utilizará un microcontrolador (ESP32 o ATmega328P), un driver MOSFET como el IRFZ44N (o equivalente) para gobernar el electroimán, así como finales de carrera para homing y elementos de señalización visual y sonora mediante LEDs y un buzzer de estado.

El software incluirá un firmware desarrollado en C/C++ y una interfaz gráfica en Python, implementada en Tkinter o PyQt. Esta ofrecerá un modo manual y un modo automático, además de control ON/OFF del electroimán y una cinemática inversa planar básica para la manipulación de piezas.

La alimentación del sistema se resolverá con una fuente capaz de abastecer tanto la lógica como la potencia, incorporando convertidores adecuados para los servos y el electroimán.

Finalmente, en el componente de documentación y docencia, se elaborará un manual de armado, una guía docente y un conjunto mínimo de cinco prácticas de laboratorio orientadas a la enseñanza y experimentación con el brazo robótico.

3.2. Criterios de aceptación

El dispositivo debe ser capaz de ejecutar trayectorias punto a punto tanto en modo manual como en modo automático. Asimismo, deberá alcanzar una tasa de éxito superior al 90 % en las tareas de agarre y manipulación de objetos, garantizando un desempeño confiable durante su operación.

De igual forma, contará con una rutina de referencia operativa que asegure la correcta inicialización del sistema, un mecanismo de paro de emergencia y límites mecánicos verificados para preservar la seguridad del equipo y del entorno.

Finalmente, el proyecto se entregará con un prototipo completamente ensamblado, acompañado del firmware correspondiente, la interfaz gráfica de usuario, un manual de usuario detallado y una guía docente que incluya cinco prácticas evaluables para su aplicación en el ámbito académico.

3.3. Segmentación de proyecto

1. **Métodos de fabricación:** Definir el flujo de fabricación y los materiales del prototipo. Priorizar MDF cortado a láser para iteraciones rápidas y precisión 2D, y emplear PLA impreso en 3D en interfaces sometidas a fricción o geometrías no realizables a láser. Diseñar y fabricar galgas para validar tolerancias y encastrés MDF-MDF, PLA-PLA y mixtos. Documentar parámetros de corte e impresión y criterios de aceptación dimensional.

2. **Diseño y manufactura:** Modelar la estructura del brazo considerando distribución de masas, trayectorias y rigidez. Dimensionar y ubicar servomotores (MG996R y SG90), soportes, eslabones y topes mecánicos; definir uniones, casquillos y separadores. Fabricar base, articulaciones, manipulador (garra) y controlador espejo con sus accesorios. Verificar interferencias, holguras y repetibilidad; ajustar iterativamente las piezas hasta lograr el encastre objetivo.
3. **Electrónica:** Integrar alimentación a 5 V, distribución de potencia y protección básica. Cablear Arduino UNO con servomotores, indicadores LED y buzzer; incorporar el controlador espejo como entrada de posición/estado. Trazar esquema eléctrico, asignar pines y preparar placa o protoboard de control. Validar corrientes de pico y caídas de tensión en carga.
4. **Programación:** Desarrollar el firmware del Arduino con modos de operación manual, espejo y recorrido preprogramado. Implementar comunicación serial con la interfaz de usuario, temporización y filtrado básico de señales. Incluir homing o posiciones seguras, límites de movimiento y manejo de errores. Entregar archivo fuente versionado y un set mínimo de pruebas funcionales.
5. **Documentación:** Producir el paquete de entrega: manual de uso, manual de ensamblaje y planos de fabricación. Incluir diagrama eléctrico, lista de materiales, parámetros de corte/impresión, y guías de mantenimiento. Registrar decisiones de diseño, riesgos y recomendaciones para réplica en ámbito educativo. Versionar todos los documentos y figuras con sus fuentes.

3.3.1. Diagrama de Gantt

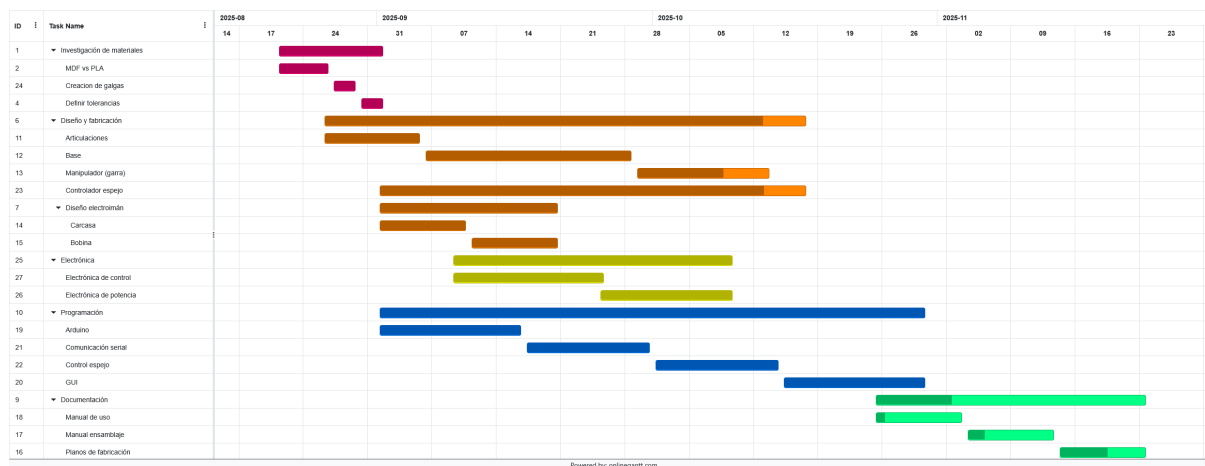


Figura 3: Diagrama de Gantt. Fuente: elaboración propia

3.4. Materiales

■ Estructura mecánica

- MDF de 3.0–3.2 mm, corte láser (placas para base, eslabones y soportes).
- Piezas de interfaz impresas en 3D (PLA) para articulaciones, casquillos/espaciadores y topes.

- Tornillería M3: tornillos (varias longitudes), tuercas, arandelas planas y de presión.
 - Adhesivos y acabado: cianoacrilato/epoxi según unión, sellador para MDF, lubricante seco para contactos PLA–PLA.
 - Bridas y organizadores de cableado (guías/ojales) para alivio de tensión.
- Actuadores
- $4 \times$ servomotores MG996R de 180° (una unidad por articulación principal).
 - $1 \times$ servomotor SG90 de 180° (garra).
 - Electroimán: bobina de alambre de cobre esmaltado (calibre acorde al diseño), núcleo de acero dulce y carcasa.
- Electrónica de control
- Plataforma de desarrollo Arduino UNO Rev3.
 - Driver para electroimán: MOSFET canal N, resistencia de compuerta ($100\ \Omega$), resistor *pull-down* ($100\ \text{k}\Omega$) y diodo de rueda libre (p. ej. 1N540x o Schottky) en la bobina.
 - Indicadores: LEDs con sus resistencias limitadoras ($220\text{--}330\ \Omega$) y buzzer 5 V.
 - $4 \times$ potenciómetros lineales $10\ \text{k}\Omega$ para el controlador espejo (con perillas y cables).
 - Distribución de potencia: borneras atornillables o conectores eléctricos, portafusibles y fusibles por rama de servos ($3\text{--}5\ \text{A}$ típicos), condensadores electrolíticos de desacople ($1000\text{--}2200\ \mu\text{F}$ cerca de grupos de servos) y cerámicos locales.
 - Cableado de potencia y señal (sección acorde: p. ej. AWG18–20 para 5 V de servos), funda termorretráctil e interruptor general.
 - Protoboard o placa perforada para interconexión y fijación de componentes.
- Fuente de alimentación
- Fuente ATX de PC (uso del riel +5 V). Adaptador/breakout a bornes, puenteo PS_ON a GND para encendido y masa en estrella.
 - (Opcional para pruebas) fuente conmutada 5 V–10 A tipo “LED”.

3.5. Métodos de fabricación

3.5.1. Tolerancias de fabricación

Se fabrican galgas con impresión 3D y corte láser para identificar tolerancias y ajustes durante el proceso de diseño y fabricación.



Figura 4: Galgas para medición de tolerancias. Fuente: elaboración propia

Basado en pruebas de encaje utilizando las galgas fabricadas, los ajustes para impresión son:

- **Ajuste apretado:** 0.15mm a 0.20mm
- **Ajuste deslizante:** 0.25mm a 0.30mm
- **Ajuste flojo:** 0.30mm en adelante

Para imprimir superficies con interferencia entre sí, se debe considerar la presencia de la costura de impresión. Al momento de diseñar se debe incluir un receso en superficies redondas para permitir que la costura no sobresalga al momento de la impresión, y perjudicar la interferencia deseada.

El corte láser brinda una precisión prácticamente exacta en la capa superficial donde se encuentra el punto focal del láser. Se debe considerar que mientras el láser atraviesa el material, y la superficie de contacto con el láser se va alejando del punto de foco, se produce una apertura de aproximadamente 1.8 grados. Lo que se traduce en que si se diseña un agujero de 10mm, en la cara trasera habrán 10.2mm.

3.6. Diseño y fabricación

3.6.1. Articulaciones desmontables

Con base en los lineamientos de tolerancias del apartado 3.5.1, se diseñó y fabricó una articulación desmontable para el brazo robótico. El objetivo fue verificar encastres, juego angular y comportamiento frente a rozamiento en las superficies de contacto.

Inicialmente se construyó con descartes de MDF cortados a láser y herrajes M3. La pieza se sometió a esfuerzos de fricción, corte y rotación para observar desgaste, holguras y deformaciones.

El prototipo permitió validar el principio mecánico, pero evidenció pérdida de tolerancia en contactos MDF–MDF bajo uso repetido.

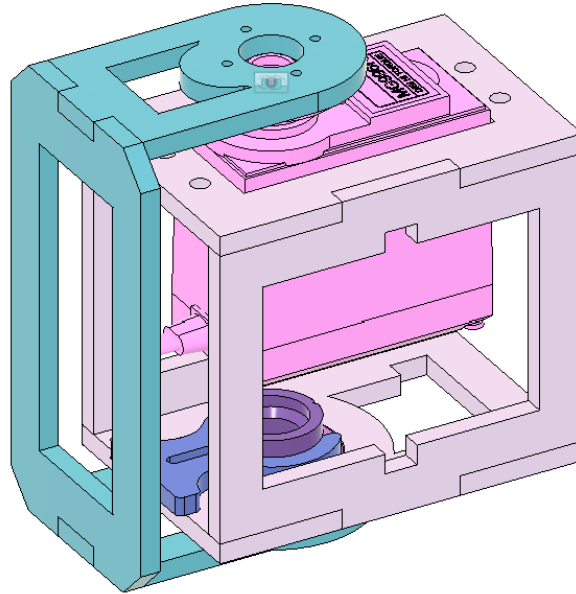


Figura 5: Sistema de articulación desmontable. Fuente: elaboración propia

Posteriormente se reemplazaron las superficies de contacto por insertos impresos en PLA, configurando una interfaz PLA–PLA en las zonas con rozamiento. Se ajustaron tolerancias según las galgas obtenidas, reduciendo el juego en el conjunto ensamblado y mejorando la repetibilidad. Se documentaron espesores, holguras objetivo y par de apriete de tornillería para su reproducción. Véase figura 6

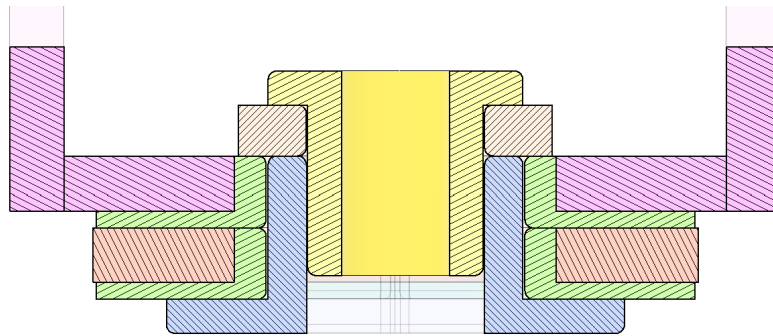


Figura 6: Corte transversal perno — versión mejorada. Fuente: elaboración propia

Se identifican con verde las superficie de contacto que friccionan contra el perno azul, las cuáles tienen un ajuste deslizante con el perno. En amarillo se muestra la pieza que actúa como tope para el seguro del perno. En el modelo anterior se utilizaba una sola pieza que hacía esta función, pero al utilizar este tope (con un ajuste apretado) permite regular la fuerza de ajuste y juego de la articulación. Luego el seguro que se encuentra entre el tope y el perno, permite

montar y desmontar la articulación de manera sencilla.

3.6.2. Electroimán

Para realizar la carcasa del electroimán se realiza uso de un torno manual. Hasta tener las medidas concretadas, se dejan un excedente en espesor y longitud para realizar pruebas durante el proceso de prototipado. Cuando se obtengan el resto de valores se dará lugar a una finalización de esta pieza por motivos de reducción de peso.

La bobina del electroimán se construyó utilizando alambre de cobre esmaltado de 0,5 mm de diámetro, enrollado a lo largo de aproximadamente 5 metros para generar el campo magnético necesario. Como núcleo ferromagnético se empleó un tornillo de acero, seleccionado por su alta permeabilidad magnética y disponibilidad.

Para asegurar la integridad mecánica del devanado y permitir un montaje limpio dentro del efector, se diseñó e imprimió en 3D una carcasa encapsulante, la cual facilita el proceso de bobinado, protege el alambre y permite fijar la bobina de manera estable en la estructura del electroimán. Este encapsulamiento también mejora la manipulabilidad del conjunto y reduce el riesgo de daños por fricción o tensiones mecánicas durante la operación del brazo robótico.



Figura 7: Pieza fabricada. Fuente: elaboración propia

3.7. Fuente de alimentación

En lo referente a la electrónica de potencia, se utilizó una fuente ATX de una computadora en desuso para alimentar los motores del sistema. Esta fuente fue seleccionada debido a su capacidad para suministrar líneas estables de 12 V y 5 V, necesarias para los distintos actuadores. Además, se diseñó e imprimió una carcasa para alojar y proteger la fuente, permitiendo un montaje seguro y una mejor organización del cableado dentro del proyecto, la misma se enciende por un interruptor ubicado en la parte delantera de la carcasa.

3.7.1. Controlador espejo

Se realiza el diseño del controlador espejo tomando en consideración las tolerancias especuladas. Se elaboran bosquejos y se diseñan partes individuales para verificar calces y encastrés, tomando como inspiración el anexo 18.

3.8. Código microcontrolador

3.9. Electrónica de control

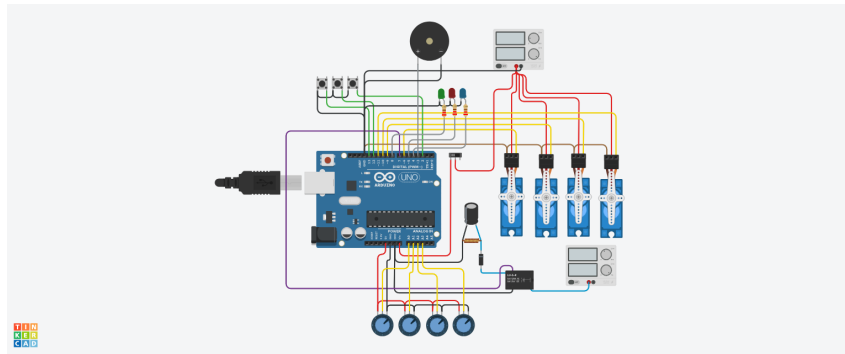


Figura 8: Diagrama de conexiones eléctricas. Fuente: elaboración propia

3.10.

Referencias

- Arduino. (2024a). *Arduino nano — tech specs*. Descargado de <https://store-usa.arduino.cc/products/arduino-nano> (Dimensiones, pines y consumo del Arduino Nano; consultado: 2025-09-27)
- Arduino. (2024b). *Uno r3 / arduino documentation*. Descargado de <https://docs.arduino.cc/hardware/uno-rev3> (Especificaciones oficiales del Arduino UNO; consultado: 2025-09-27)
- Components101. (2025). *MG996R High Torque Metal Gear Dual Ball Bearing Servo*. Descargado de https://components101.com/sites/default/files/component_datasheet/MG996R%20Datasheet.pdf
- Espressif Systems. (2024a). *Esp32 series — datasheet*. Descargado de https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf (Características generales, PWM, GPIO matrix; consultado: 2025-09-27)
- Espressif Systems. (2024b). *Esp32 technical reference manual*. Descargado de https://www.espressif.com/sites/default/files/documentation/esp32_technical

- _reference_manual_en.pdf (Detalles eléctricos a 3.3 V y periféricos; consultado: 2025-09-27)
- Fouly, A., y cols. (2022). Evaluating the mechanical and tribological properties of 3d-printed polymers: A review. *Polymers*, 14(24), 5423. Descargado de <https://pmc.ncbi.nlm.nih.gov/articles/PMC9738854/> (Open access; Consultado: 2025-09-27) doi: 10.3390/polym14245423
- GrabCAD. (2021a). *6dof robot arm*. Descargado de <https://grabcad.com/library/6dof-robot-arm-1>
- GrabCAD. (2021b). *Robotic arm 216*. Descargado de <https://grabcad.com/library/robotic-arm-216>
- GrabCAD. (2021c). *Robotic gripper 17*. Descargado de <https://grabcad.com/library/robotic-gripper-17>
- GrabCAD. (2021d). *Sea reach: A scalable deep sea robotic arm*. Descargado de <https://grabcad.com/library/sea-reach-a-scalable-deep-sea-robotic-arm-1>
- HyperPhysics. (s.f.). *Temperature coefficient of resistance*. Descargado 2025-09-28, de <http://hyperphysics.phy-astr.gsu.edu/hbase/electric/restmp.html> (Coeficiente típico del cobre)
- Kukla, M., Wargula, Ł., y Biszczanik, A. (2021). Determining the coefficient of friction of wood-based materials for furniture panels in the aspect of modelling their shredding process. *Wood Research*, 66(5), 789–805. Descargado de <https://www.woodresearch.sk/cms/determining-the-coefficient-of-friction-of-wood-based-materials-for-furniture-panels-in-the-aspect-of-modelling-their-shredding-process/> (Incluye valores altos de fricción para MDF no laminado; Consultado: 2025-09-27)
- Lab, M. (2022). *Robot arm mirror control printed model*. Descargado de <https://www.youtube.com/watch?v=5toNqaGsGYs>
- Libre, M. (2022). *Kit robot brazo arduino*. Descargado de https://www.mercadolibre.com.uy/kit-robot-brazo--arduino-con-garra-electronica-manual-x-pc/up/MLUU2434320874#polycard_client=search-nordic&search_layout=stack&position=2&type=product&tracking_id=5621c8d5-293c-49de-ae85-3478bbfff815&wid=MLU455624003&sid=search
- Murase, Y., y Nishino, Y. (1980). Friction of wood sliding on various materials. *Bulletin of the University Forests, Kyushu University*, 147–170. Descargado de https://api.lib.kyushu-u.ac.jp/opac_download_md/23785/p147.pdf (Influencia de la humedad y contracara; Consultado: 2025-09-27)
- OpenStax. (2016). *10.5 calculating moments of inertia*. Descargado 2025-09-28, de <https://openstax.org/books/university-physics-volume-1/pages/10-5-calculating-moments-of-inertia>
- OpenStax. (2020). *19.4 electric power*. Descargado 2025-09-28, de <https://openstax.org/books/physics/pages/19-4-electric-power>
- OpenStax. (2022). *21.3 kirchhoff's rules*. Descargado 2025-09-28, de <https://openstax.org/books/college-physics-2e/pages/21-3-kirchhoffs-rules>
- Oracid. (2013). *Mg996r servo test - 6.5kg torque at 1.6cm*. Descargado de <https://www.youtube.com/watch?v=EFnt4Ca5MSE> (Accessed: 2025-09-27)
- Physics LibreTexts. (2023). *7.2: Torque*. Descargado 2025-09-28, de https://phys.libretexts.org/Courses/Merrimack_College/Conservation_Laws_Newton%27s_Laws_and_Kinematics_version_2.0/07%3A_C7%29_Conservation_of_Angular_Momentum_II/7.02%3A_Torque
- Physics LibreTexts. (2025). *9.4: Resistivity and resistance*. Descargado 2025-09-28, de https://phys.libretexts.org/Bookshelves/University_Physics/

- University_Physics_%28OpenStax%29/University_Physics_II_-_Thermodynamics_Electricity_and_Magnetism_%28OpenStax%29/09%3A_Current_and_Resistance/9.04%3A_Resistivity_and_Resistance
- Robodacta. (2022a). *Kit mini brazo robótico con joysticks mdf*. Descargado de <https://robodacta.com.mx/producto/kit-mini-brazo-robotico-con-joysticks-mdf/>
- Robodacta. (2022b). *Robot arm mdf model*. Descargado de https://www.youtube.com/watch?v=5o_VI7fw6D8
- Ross, R. J., y Laboratory, F. P. (2021). *Wood handbook: Wood as an engineering material* (Inf. Téc.). U.S. Department of Agriculture, Forest Products Laboratory. Descargado de <https://research.fs.usda.gov/treesearch/62200> (Consultado: 2025-09-27)
- ServoCity. (s.f.). *Servo faqs*. Descargado 2025-09-28, de <https://www.servocity.com/servo-faqs> (Timing típico de servos RC: 1-2 ms sobre 20 ms)
- TechZone. (2021). *Robot arm control with potentiometers*. Descargado de <https://www.youtube.com/watch?v=ADJGx0rEZAM>
- Thang010146. (2021). *Robot arm link*. Descargado de <https://www.youtube.com/watch?v=EnAk36jC7bw>
- Thirunavukkarasu, N., y cols. (2022). Frictional behaviors of 3d-printed polylactic acid (pla). *Journal of Tribology*, 145(1), 011803. Descargado de <https://asmedigitalcollection.asme.org/tribology/article/145/1/011803/1147208/Frictional-Behaviors-of-3D-Printed-Polylactic-Acid> (Consultado: 2025-09-27) doi: 10.1115/1.4055362
- Tower Pro. (2014). Sg90 micro servo datasheet [Manual de software informático]. Descargado de <https://www.friendlywire.com/projects/ne555-servo-safe/SG90-datasheet.pdf> (Accedido: 2025-09-06)
- Tower Pro. (2015). Mg996r high-torque digital servo datasheet [Manual de software informático]. Descargado de https://www.electronicoscaldas.com/datasheet/MG996R_Tower-Pro.pdf (Accedido: 2025-09-06)
- TSE, M. (2021). *Sg90 servo test - load torque measurement*. Descargado de https://www.youtube.com/watch?v=PEcW4ja_gIE (Accessed: 2025-09-27)
- Wikipedia contributors. (2025a). *Linear interpolation*. Descargado 2025-09-28, de https://en.wikipedia.org/wiki/Linear_interpolation
- Wikipedia contributors. (2025b). *Magnetic pressure*. Descargado 2025-09-28, de https://en.wikipedia.org/wiki/Magnetic_pressure (Usado para brecha de aire)
- Xometry. (2022). *3d printing vs. laser cutting: Differences and comparison*. Descargado de <https://www.xometry.com/resources/3d-printing/3d-printing-vs-laser-cutting/> (Resumen: el corte láser es muy rápido para piezas 2D; Consultado: 2025-09-27)
- Zaitronics. (s.f.). *Exploring sg90, mg90s and mg996r servos*. Descargado 2025-09-28, de <https://zaitronics.com.au/blogs/guides/exploring-sg90-mg90s-and-mg996r-servos> (Imagen de servomotores SG90, MG90S y MG996R.)

A. Apendice

A.1. Recursos del proyecto

A.1.1. Google Drive

Se utiliza una carpeta compartida en Google Drive para almacenar documentos de trabajo, capturas de pruebas, versiones exportadas de planos e imágenes para el informe. El acceso es restringido al equipo y la docencia.

Enlace: drive.google.com/.../1XPbaXmohB05XQhgipt-RZ1E9vsX4483.

A.1.2. Repositorio en GitHub

El control de versiones del código, modelos y archivos de configuración se gestiona en GitHub. El repositorio incluye historial de cambios, issues y documentación técnica mínima para reproducir el entorno.

Enlace: github.com/HectorPereira/proyectoIntegrador2.

A.2. Inspiraciones

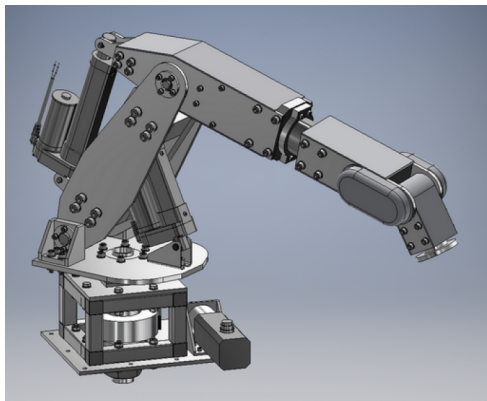


Figura 9: Modelo con actuadores lineales GrabCAD (2021a).

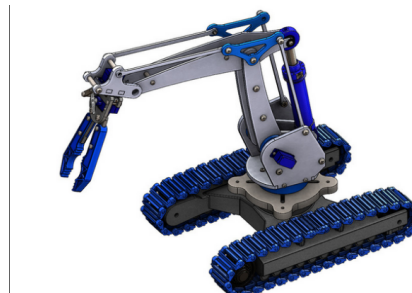


Figura 10: Modelo híbrido/hidráulico GrabCAD (2021b).

Inspiración inicial tomada de GrabCAD para un modelo con actuadores actuadores lineales. Se observa como se posicionarían los actuadores dentro del modelo para lograr una mejor distribución de masa.

Modelo mixto con servomotores y actuadores lineales encontrado en GrabCAD, primera consideración para el uso de eslabones y articulaciones. En este modelo se aprecia la restricción del movimiento de la articulación final del manipulador.

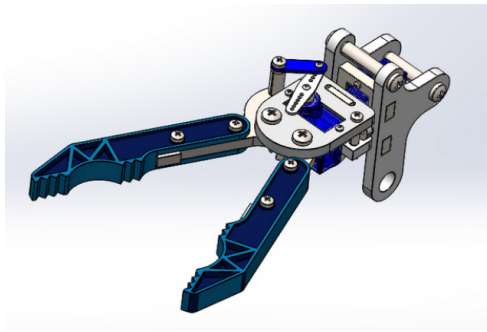


Figura 11: Manipulador por garra y servomotor GrabCAD (2021c).

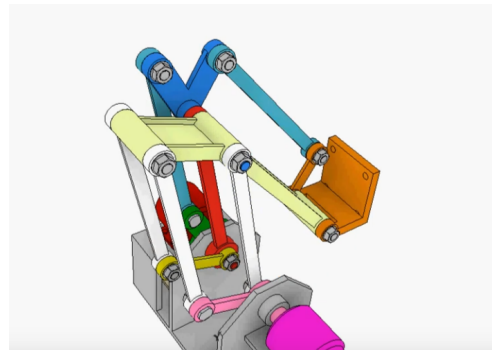


Figura 13: Sistema de eslabones con mayor libertad Thang010146 (2021).

Modelo aparte del manipulador con el servomotor sg90 y la garra. A partir de este modelo se considera el uso de un servomotor más pequeño para aprovechar que el manipulador no requiere tanta fuerza de agarre y delegar los servomotores más potentes mg996r a realizar el movimiento de las articulaciones.

Una idea más general del posicionamiento de los eslabones proporcionado por el creador de contenido ‘thang010146’ en YouTube. Este modelo cuenta con el grado de libertad faltante en modelos anteriores

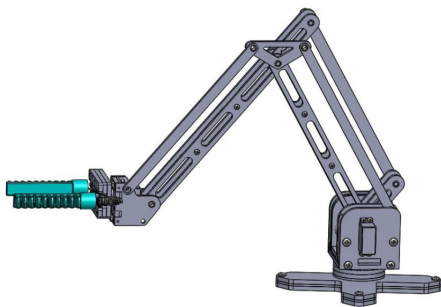


Figura 12: Sistema de eslabones GrabCAD (2021d).

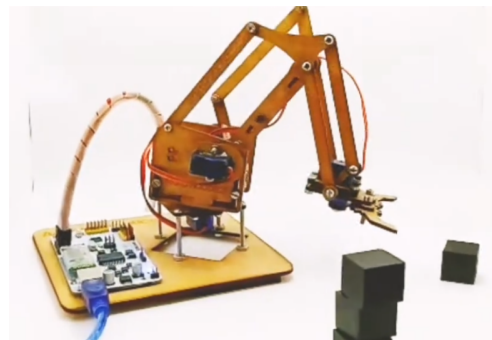


Figura 14: Modelo en MDF Robodacta (2022b).

Otra inspiración para implementar el uso de eslabones en el brazo robótico ya con el uso de servomotores mg996r (encontrado en GrabCAD)

Implementación de un brazo robótico con especificaciones similares a las buscadas y haciendo uso de materiales como MDF y tornillos. Se descubre que el modelo pertenece a una empresa Mexicana llamada Robodacta, y que distribuyen este brazo como un kit de aprendizaje ensamblable.



Figura 15: Modelo similar en plástico Libre (2022).

Modelo muy similar al anterior, haciendo uso de materiales plásticos y tornillos, comercializado en Uruguay por Mercado Libre.

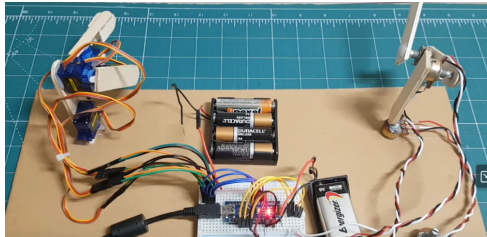


Figura 16: Controlador espejo con potenciómetros TechZone (2021)

Inspiración dedicada a la realización de un control 'espejo' para comandar el brazo articulado, haciendo uso de potenciómetros los cuales controlan el ángulo de cada articulación.

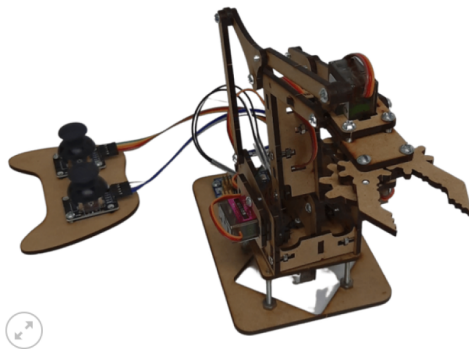


Figura 17: Control por joystick Robodacta (2022a)

Modelo original de Robodacta, el cual se encuentra publicado en su sitio web. Este cuenta con un control de joystick para comandar el brazo articulado.



Figura 18: Modelo impreso con control espejo Lab (2022)

Modelo impreso de código libre por el creador de contenido 'Build Some Stuff' con un sistema de control espejo similar al mostrado anteriormente. Dispone de una serie de videos mostrando materiales, ensamblaje, software y uso del brazo articulado con su controlador.

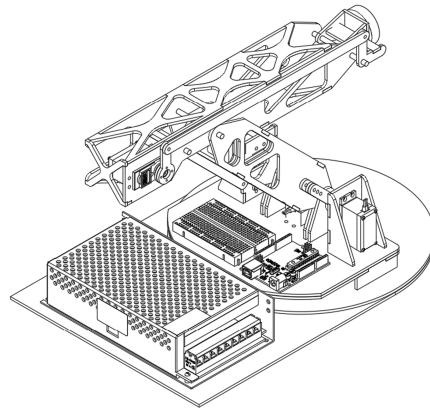


Figura 19: Diseño con eslabones y servomotores mg996r (Fuente: elaboración propia)

Diseño propio descartado. Elaborado con servomotores mg996r y eslabones como prueba de concepto preliminar.

A.3. Código para el microcontrolador

A.3.1. Módulo UART: Inicialización y telemetría de servos

```
1 void uart_init(void) {
2     UBRROH = (uint8_t)(UART_UBRR >> 8);
3     UBRROL = (uint8_t)(UART_UBRR & 0xFF);
4
5     UCSROB = (1 << RXEN0) | (1 << TXEN0);    // RX/TX habilitados
6     UCSROC = (1 << UCSZ01) | (1 << UCSZ00);    // 8N1
7 }
8
9 void uart_send_positions(void) {
10     char buf[48];
11
12     unsigned s1 = angle_logical[0];
13     unsigned s2 = angle_logical[1];
14     unsigned s3 = 180 - angle_logical[2];    // inversión para la app
15     unsigned s4 = angle_logical[3];
16
17     int n = snprintf(buf, sizeof(buf),
18         "S1=%u S2=%u S3=%u S4=%u\n",
19         s1, s2, s3, s4);
20
21     if (n > 0) uart_send_str(buf);
22 }
```

A.3.2. Parser de comandos UART (MODE, MAG, GON/GOFF, S1..S4)

```
1 void parse_uart_line(char *s) {
2     while (*s) {
3         while (*s=='\n' || *s=='\t') s++;
4
5         // MODE:APP / MODE:CE
6         if (strncmp(s,"MODE",4)==0) {
7             s += 4;
8             while (*s=='\n' || *s=='\t' || *s==':' || *s=='=') s++;
9             if (*s=='A' || *s=='a') current_mode = MODE_APP;
10            else current_mode = MODE_CE;
11            while(*s && *s!='\n' && *s!='\t') s++;
12            continue;
13        }
14
15        // MAG:0 / MAG:1
16        if (strncmp(s,"MAG",3)==0) {
17            s += 3;
18            while (*s=='\n' || *s=='\t' || *s==':' || *s=='=') s++;
19            int val = atoi(s);
20            mag_set(val ? 1 : 0);
21            while(*s && *s!='\n' && *s!='\t') s++;
22            continue;
23        }
24
25        // GON / GOFF (grabación)
```

```

26     if (strncmp(s,"GON",3)==0) {
27         rec_set(1, 0);
28         s += 3;
29         continue;
30     }
31     if (strncmp(s,"GOFF",4)==0) {
32         rec_set(0, 0);
33         s += 4;
34         continue;
35     }
36
37     // S1..S4
38     if (*s=='S' && (s[1]>='1'&&s[1]<='4')) {
39         uint8_t idx = s[1]-'1';
40         s += 2;
41         while(*s=='_' || *s=='\t' || *s==':' || *s=='=') s++;
42         int val = atoi(s);
43         if (val<0) val = 0;
44         if (val>180) val = 180;
45         angle_app[idx] = val;
46         while(*s && *s!='_' && *s!='\t') s++;
47         continue;
48     }
49
50     while(*s && *s!='_' && *s!='\t') s++;
51 }
52 }

```

A.3.3. Algoritmo de suavizado incremental (SLEW) para los servos

```

1  for(uint8_t i=0; i<4; i++){
2      long logical_angle;
3
4      // 1) Ángulo objetivo según el modo (CE: joystick / APP: UART)
5      if (current_mode == MODE_CE) {
6          uint16_t val = ADC_read(potPins[i]);
7          long pot_a = map_value(val,0,1023,0,270);
8          long off = map_value(potCenter[i],0,1023,0,270);
9          logical_angle = 90 + (pot_a - off);
10     } else {
11         logical_angle = angle_app[i];
12     }
13
14     if (logical_angle < 0) logical_angle = 0;
15     if (logical_angle > 180) logical_angle = 180;
16     angle_logical[i] = logical_angle;
17
18     // 2) Suavizado SLEW: angle_output = angle_logical
19     int16_t out = angle_output[i];
20
21     if (out < logical_angle) {
22         out += SLEW_STEP_DEG;
23         if (out > logical_angle) out = logical_angle;
24     } else if (out > logical_angle) {

```

```

25     out -= SLEW_STEP_DEG;
26     if (out < logical_angle) out = logical_angle;
27 }
28
29 angle_output_prev[i] = angle_output[i];
30 angle_output[i] = out;
31 }

```

A.3.4. Generación de PWM por software (bit-banging) usando Timer1

```

1 void servo_pulse_frame(void){
2     // Subir los cuatro pines simultáneamente
3     PORTD |= (1<<PD6);
4     PORTB |= (1<<PB1)|(1<<PB2)|(1<<PB3);
5
6     uint16_t start = TCNT1;
7
8     for(;;){
9         uint16_t now = TCNT1;
10        uint16_t us = (now - start) / 2;    // 0.5 us por tick
11
12        uart_poll_byte();    // se permite recepción UART dentro del frame
13
14        if (us >= SERVO_PERIOD_US) break;
15
16        if (us >= pulseWidth[0]) PORTD &= ~(1<<PD6);
17        if (us >= pulseWidth[1]) PORTB &= ~(1<<PB1);
18        if (us >= pulseWidth[2]) PORTB &= ~(1<<PB2);
19        if (us >= pulseWidth[3]) PORTB &= ~(1<<PB3);
20    }
21 }

```

A.3.5. Control del electroimán (MAG) con feedback mediante buzzer

```

1 void mag_set(uint8_t new_state) {
2     uint8_t old = mag_state;
3     mag_state = new_state ? 1 : 0;
4     mag_apply_state();
5
6     if (!old && mag_state) {
7         // Si pasó de OFF -> ON, 2 pitidos
8         buzzer_start(2);
9     }
10 }
11
12 void mag_button_poll(void) {
13     uint8_t now = (PINB & (1<<MAG_BTN_PIN)) ? 1 : 0;
14
15     if (mag_btn_last == 1 && now == 0 && mag_btn_debounce == 0) {
16         mag_set(mag_state ? 0 : 1);
17
18         if (mag_state) uart_send_str("MAG:1\n");

```

```

19         else                uart_send_str("MAG:0\n");
20
21         mag_btn_debounce = 10;
22     }
23
24     mag_btn_last = now;
25 }

```

A.3.6. Máquina de estados del buzzer para patrones de beeps

```

1 void buzzer_start(uint8_t n_beeps) {
2     if (n_beeps == 0) return;
3     buzzer.active      = 1;
4     buzzer.beeps_total = n_beeps;
5     buzzer.beeps_done  = 0;
6     buzzer.phase       = 1;      // ON
7     buzzer.phase_ticks = 0;
8     PORTD |= (1 << BUZZER_PIN);
9 }
10
11 void buzzer_update(void) {
12     if (!buzzer.active) {
13         PORTD &= ~(1 << BUZZER_PIN);
14         return;
15     }
16
17     buzzer.phase_ticks++;
18
19     if (buzzer.phase == 1) {      // fase ON
20         if (buzzer.phase_ticks >= BUZZ_ON_TICKS) {
21             buzzer.phase = 0;
22             buzzer.phase_ticks = 0;
23             PORTD &= ~(1 << BUZZER_PIN);
24         }
25     } else {                      // fase OFF
26         if (buzzer.phase_ticks >= BUZZ_OFF_TICKS) {
27             buzzer.beeps_done++;
28             if (buzzer.beeps_done >= buzzer.beeps_total) {
29                 buzzer.active = 0;
30             } else {
31                 buzzer.phase = 1;
32                 buzzer.phase_ticks = 0;
33                 PORTD |= (1 << BUZZER_PIN);
34             }
35         }
36     }
37 }

```

A.3.7. Antirrebote por software de los botones físicos

```

1 void mode_button_poll(void) {
2     uint8_t now = (PIND & (1<<BTN_MODE_PIN)) ? 1 : 0;

```

```

3
4     if (btn_last == 1 && now == 0 && btn_debounce == 0) {
5         current_mode = (current_mode == MODE_CE) ? MODE_APP : MODE_CE;
6         uart_send_str(current_mode == MODE_APP ? "MODE=APP\n" : "MODE=CE\n");
7         btn_debounce = 10;
8     }
9     btn_last = now;
10 }

```

A.3.8. Bucle principal del firmware (control, PWM, telemetría)

```

1 while(1){
2     mode_button_poll();
3     mag_button_poll();
4     rec_button_poll();
5
6     uint8_t moving = 0;
7
8     // Actualización lógica y suavizada de los cuatro servos
9     for(uint8_t i=0; i<4; i++){
10         // Cálculo del ángulo lógico según modo CE/APP
11         ...
12         // Suavizado: angle_output se acerca a logical_angle
13         ...
14         // Conversión a ángulo físico del servo y PWM
15         pulseWidth[i] = SERVO_MIN_US +
16             (servo_angle*(SERVO_MAX_US-SERVO_MIN_US))/180;
17     }
18
19     // Generación de PWM para los 4 servos
20     servo_pulse_frame();
21
22     // Actualización de buzzer y LED verde
23     buzzer_update();
24     led_green_update(moving);
25
26     // Telemetría periódica hacia la PC
27     tcount++;
28     if (tcount >= 5){
29         tcount = 0;
30         uart_send_positions();
31     }
32 }

```

A.4. Código para la interfaz de usuario

A.4.1. Módulo de conexión y comunicación serial (UART)

```

1 def connect_serial():
2     global ser, connected
3     port = port_combo.get().strip()
4     ser = serial.Serial(port, BAUD, timeout=0.2)

```

```

5     time.sleep(0.4)    # Reset automático del Arduino
6     connected = True
7     periodic_read()    # Arranca ciclo de lectura

```

```

1 def periodic_read():
2     if not connected:
3         return
4     data = read_available().strip()
5     if data:
6         parse_and_update(data)
7     root.after(READ_POLL_MS, periodic_read)

```

A.4.2. Módulo de interpretación de mensajes del firmware (Parser UART)

```

1 def parse_and_update(text: str):
2     for line in text.splitlines():
3         line = line.strip()
4         if not line:
5             continue
6         upper = line.upper()
7
8         # Inicio/fin de grabación (botón físico en el robot)
9         if upper.startswith("GON"):
10             start_record_from_hw()
11             continue
12         if upper.startswith("GOFF"):
13             stop_record_from_hw()
14             continue
15
16         # Lectura del electroimán: "MAG:1", "MAG=0", etc.
17         if upper.startswith("MAG"):
18             tmp = upper.replace("=", "_").replace(":", "_")
19             parts = tmp.split()
20             if len(parts) >= 2:
21                 mag_state = 1 if parts[1] in ("1", "ON") else 0
22                 mag_btn.config(text=f"Electroimán: {'ON' if mag_state else 'OFF'}")
23                 update_hud()
24                 continue
25
26         # Telemetría completa de servos: "S1=90 S2=120 S3=45 S4=30"
27         if ("S1=" in line) and ("S2=" in line) and ("S3=" in line) and ("S4=" in line):
28             parts = line.split()
29             for tok in parts:
30                 if tok.startswith("S") and "=" in tok:
31                     name, val = tok.split("=", 1)
32                     set_servo_angle(name, int(val))
33             update_hud()

```

A.4.3. Envío progresivo de comandos desde los sliders (Rate Limiter)

```

1 def _send_pending_for(key: str):
2     if is_centering:
3         servos[key]["after_id"] = None
4         return
5
6     deg = int(servos[key]["pending"])
7     lo, hi = servos[key]["min"], servos[key]["max"]
8     deg = max(lo, min(hi, deg))
9     servos[key]["disp"].set(str(deg))
10
11     # Envío controlado para evitar saturación por UART
12     if mode_var.get() == "APP" and deg != servos[key]["last_sent"]:
13         send_deg = transform_angle_for_send(key, deg)
14         write_line(f"{key}:{send_deg:d}")
15         servos[key]["last_sent"] = deg
16
17     if connected and mode_var.get() == "APP":
18         servos[key]["after_id"] = root.after(SEND_INTERVAL_MS,
19                                             _send_pending_for, key)
20
21     update_hud()

```

A.4.4. Algoritmo de centrado suave de los cuatro servos

```

1 def center_all_smooth():
2     global is_centering
3     if is_centering:
4         return
5
6     is_centering = True
7     set_scales_state("disabled")
8
9     targets = {}
10    for k in ("S1", "S2", "S3", "S4"):
11        lo, hi = servos[k]["min"], servos[k]["max"]
12        targets[k] = max(lo, min(hi, 90))
13
14    def _tick():
15        nonlocal targets
16        global is_centering
17
18        all_done = True
19        for k in ("S1", "S2", "S3", "S4"):
20            cur = int(round(servos[k]["var"].get()))
21            tgt = targets[k]
22            if cur != tgt:
23                all_done = False
24                step = CENTER_STEP_DEG
25                cur = cur + step if cur < tgt else cur - step
26                set_servo_angle(k, cur)
27
28        if connected and mode_var.get() == "APP":
29            cmd = " ".join(f"{k}:{transform_angle_for_send(k, int(servos
30                                [k]['var'].get()))}"

```



```

30         for k in ("S1", "S2", "S3", "S4"))
31             write_line(cmd)
32
33     update_hud()
34
35     if all_done:
36         is_centering = False
37         set_scales_state("normal")
38         last_msg_var.set("Último: _Centrado _suave _completado")
39     else:
40         root.after(CENTER_INTERVAL_MS, _tick)
41
42     _tick()

```

A.4.5. Grabación periódica de la trayectoria (Logger de poses)

```

1 def record_tick():
2     global record_after_id
3     if not recording:
4         record_after_id = None
5         return
6
7     frame = {}
8     for k in ("S1", "S2", "S3", "S4"):
9         frame[k] = int(round(servos[k]["var"].get()))
10
11     # Incluir estado del electroimán
12     frame["MAG"] = int(mag_state)
13
14     recorded_frames.append(frame)
15     record_after_id = root.after(RECORD_INTERVAL_MS, record_tick)

```

A.4.6. Reproducción de trayectorias con prealineado suave

```

1 def smooth_move_to_frame(frame, done_callback):
2     global preplay_active, is_centering
3     preplay_active = True
4     is_centering = True
5     set_scales_state("disabled")
6
7     targets = {k: int(frame.get(k, servos[k]["var"].get()))
8                 for k in ("S1", "S2", "S3", "S4")}
9
10    def _tick():
11        nonlocal targets
12        all_done = True
13
14        for k in ("S1", "S2", "S3", "S4"):
15            cur = int(round(servos[k]["var"].get()))
16            tgt = targets[k]
17            if cur != tgt:
18                all_done = False

```

```

19         cur += CENTER_STEP_DEG if cur < tgt else -
20           CENTER_STEP_DEG
21         set_servo_angle(k, cur)
22
23     if connected and mode_var.get() == "APP":
24         cmd = " ".join(
25             f"{k}:{transform_angle_for_send(k,int(servos[k]['var']).
26               get())}"
27             for k in ("S1","S2","S3","S4")
28         )
29         write_line(cmd)
30
31     update_hud()
32
33     if all_done:
34         preplay_active = False
35         is_centering = False
36         set_scales_state("normal")
37         done_callback()
38     else:
39         root.after(CENTER_INTERVAL_MS, _tick)

```

```

1 def start_playback():
2     global is_playing, play_index
3
4     if not recorded_frames or not connected:
5         return
6
7     first_frame = recorded_frames[0]
8
9     def after_align():
10         nonlocal first_frame
11         is_playing = True
12         play_index = 0
13         playback_step()
14
15     smooth_move_to_frame(first_frame, after_align)

```

A.4.7. Paso discreto de reproducción de la trayectoria

```

1 def playback_step():
2     global is_playing, play_index, play_after_id, mag_state
3
4     if not is_playing:
5         return
6
7     if play_index >= len(recorded_frames):
8         is_playing = False
9         return
10
11     frame = recorded_frames[play_index]
12
13     # 1) Actualizar sliders

```

```

13     for k in ("S1", "S2", "S3", "S4"):
14         set_servo_angle(k, int(frame[k]))
15
16     # 2) Electroimán sincronizado
17     if "MAG" in frame:
18         mag_state = int(frame["MAG"])
19         mag_btn.config(text=f"Electroimán: {'ON' if mag_state else 'OFF'}")
20
21     # 3) Enviar al firmware
22     if connected and mode_var.get() == "APP":
23         parts = []
24         for k in ("S1", "S2", "S3", "S4"):
25             deg = int(servos[k]["var"].get())
26             parts.append(f"{k}:{transform_angle_for_send(k, deg)}")
27         parts.append(f"MAG:{mag_state}")
28         write_line("\n".join(parts))
29
30     update_hud()
31
32     play_index += 1
33     if is_playing and play_index < len(recorded_frames):
34         play_after_id = root.after(RECORD_INTERVAL_MS, playback_step)

```

A.4.8. Control del electroimán vía UART

```

1 def toggle_magnet():
2     global mag_state
3     if not connected:
4         return
5
6     mag_state = 1 - mag_state
7     mag_btn.config(text=f"Electroimán: {'ON' if mag_state else 'OFF'}")
8     write_line(f"MAG:{mag_state}")
9     update_hud()

```

A.4.9. Visualización 2D del brazo robótico (HUD cinemático)

```

1 def update_hud():
2     s2 = int(servos["S2"]["var"].get())
3     s3 = int(servos["S3"]["var"].get())
4     s4 = int(servos["S4"]["var"].get())
5
6     a1 = math.radians(s2)
7     a2 = a1 + math.radians(s3 - 90)
8     a3 = a2 + math.radians(s4 - 90)
9
10    x1 = ox + L1*cos(a1)
11    y1 = oy - L1*sin(a1)
12    x2p = x1 + L2*cos(a2)
13    y2p = y1 - L2*sin(a2)
14    x3 = x2p + L3*cos(a3)

```

```

15 arm_canvas.create_line(ox, oy, x1, y1, width=6)
16 arm_canvas.create_line(x1, y1, x2p, y2p, width=6)
17 arm_canvas.create_line(x2p, y2p, x3, y3, width=6)
18

```

A.4.10. Módulo de interpretación de mensajes del firmware (Parser UART)

```

1 def parse_and_update(text: str):
2     for line in text.splitlines():
3         upper = line.upper()
4
5         if upper.startswith("GON"):
6             start_record_from_hw()
7         elif upper.startswith("GOFF"):
8             stop_record_from_hw()
9
10        elif upper.startswith("MAG"):
11            token = upper.replace("=", " ").replace(":", " ").split()[1]
12            mag_state = 1 if token in ("1", "ON") else 0
13            mag_btn.config(text=f"Electroimán: {'ON' if mag_state else 'OFF'}")
14
15        elif "S1=" in line and "S2=" in line:
16            for tok in line.split():
17                if "S" in tok:
18                    name, val = tok.split("=")
19                    set_servo_angle(name, int(val))
20            update_hud()

```